Сессионное задание

1. **Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны.**

Если в сети много пользователей, то хранить отдельные симметричные ключи для каждой пары (всего n(n-1)/2 ключей) неудобно и сложно. Поэтому используют трехсторонние протоколы, где есть доверенный посредник - центр распределения ключей (KDC или ЦГРК).

Участники протокола и их роли:

1) Пользователь A (Алиса) - инициатор, который хочет начать защищенный обмен сообщениями с пользователем B.
2) Пользователь B (Боб) - получатель, с которым хочет связаться A.
3) Центр распределения ключей (KDC) - третья сторона, которой доверяют оба абонента. У центра заранее есть общий секретный ключ с каждым пользователем: ключ Ka (известен только A и KDC) и ключ Kb (известен только B и KDC).

Основные этапы работы протокола (на примере протокола Ниидома-Шредера из лекции):

Шаг 1. Абонент A отправляет центру KDC открытый запрос, в котором указывает свой идентификатор, идентификатор Боба и случайное число (метку) Ra: (A, B, Ra).

Шаг 2. KDC генерирует случайный сеансовый ключ Kab для текущей сессии. После этого KDC шифрует ключом Ka сообщение для Алисы, куда кладет сам сеансовый ключ Kab, имя B, метку Ra, и отдельно зашифрованный ключом Боба Kb «билет» (внутри билета лежат ключ Kab и имя отправителя A). KDC пересылает все это сообщение абоненту A.

Шаг 3. Абонент A расшифровывает внешнее сообщение своим ключом Ka, забирает себе сеансовый ключ Kab, а зашифрованный

ключом Kb билет пересылает абоненту B (прочитать или подделать этот билет A не может, т.к. не знает Kb).

Шаг 4. Абонент B получает билет, расшифровывает его своим ключом Kb и узнает сеансовый ключ Kab и то, что связь запросил абонент A. Для проверки Боб придумывает случайное число Rb, шифрует его уже новым ключом Kab и отправляет Алисе. Алиса расшифровывает его, вычитает 1 и отправляет обратно зашифрованное на Kab значение Rb - 1.

Итог работы протокола: между пользователями A и B установлен общий секретный сеансовый ключ Kab. Дальше они шифруют им свои сообщения напрямую друг другу без участия KDC, а после окончания сеанса этот ключ удаляется.

2. Приведите формальные модели шифров.

В криптографии для описания работы шифров и анализа их стойкости обычно выделяют три формальные модели:

1) Алгебраическая (детерминированная) модель шифра.

В этой модели шифр задается через множества и функции: множество открытых текстов X*, множество шифртекстов (криптограмм) Y*, ключевое множество K (конечное множество всех возможных ключей), семейство функций зашифрования E = {Ek} и семейство обратных функций расшифрования D = {Dk}. При выбранном ключе k функция Ek переводит открытый текст x в шифртекст y = Ek(x), причем отображение должно быть биективным по первой переменной, чтобы получатель мог однозначно расшифровать текст обратно: Dk(y) = x. Такая модель нужна для строгого описания правил шифрования и расшифрования, проверки обратимости шифра и деления криптосистем на симметричные (где ключ один и тот же) и асимметричные (где ключи зашифрования и расшифрования разные).

2) Вероятностная модель шифра (модель Шеннона).

Здесь к множествам открытых текстов и ключей добавляются распределения вероятностей: вероятность P(x) появления открытого текста x (зависит от статистики и частот букв языка) и вероятность P(k)

выбора ключа k из ключевого множества K (обычно считают, что все ключи выбираются равновероятно P(k) = 1/|K| и независимо от текста). Они задают распределение вероятностей криптограмм P(y) и условные апостериорные вероятности P(x|y) того, что был передан текст x, если перехвачена криптограмма y. Эта модель используется криптоаналитиками для статистического анализа шифрсистемы и оценки ее стойкости. Через нее Шеннон определил совершенно стойкий шифр - когда открытый текст и шифртекст статистически независимы, т.е. P(x|y) = P(x).

3) Автоматная модель шифра.

Описывает шифр как конечный автомат с памятью. В его состав входят входной и выходной алфавиты, множество внутренних состояний (регистры памяти), начальное состояние (ключ и вектор инициализации), функция переходов между состояниями и функция выходов. Модель применяется для описания поточных шифров и режимов блочных шифров с обратной связью или счетчиком, где шифрование следующего блока или бита зависит не только от ключа, но и от текущего состояния регистров.

3. Приведите принципы построения поточных систем шифрования.

Поточные шифры шифруют данные по одному биту или байту. Основные принципы их построения:

1) Побитовое наложение гаммы (гаммирование). Открытый текст складывается побитово с ключевым потоком (гаммой) с помощью операции исключающее ИЛИ (XOR, сложение по модулю 2): C = P XOR G, и точно так же расшифровывается обратно: P = C XOR G. Это нужно для высокой скорости работы, простоты реализации в железе и программах, шифрования «на лету» без задержек и отсутствия размножения ошибок (если в канале исказился 1 бит, то при расшифровании испортится тоже только 1 бит, а не весь текст).

2) Генерация длинной псевдослучайной гаммы из короткого ключа. В идеальном шифре Вернама ключ должен быть равен длине сообщения, что на практике очень неудобно передавать и хранить. Поэтому в поточных шифрах берут короткий секретный ключ и пропускают его через детерминированный генератор гаммы, который растягивает его в поток битов нужной длины. Этот принцип нужен для эффективности и удобства управления ключами.
3) Близость гаммы к случайной последовательности. Поскольку гамма не истинно случайная, а псевдослучайная, она должна иметь большой период и хорошие статистические свойства (равномерное распределение 0 и 1, прохождение статистических тестов), для чего часто комбинируют несколько генераторов нелинейными функциями. Это нужно для криптостойкости, чтобы злоумышленник по куску гаммы не смог вычислить ключ или угадать следующие биты.
4) Одноразовое использование гаммы (ключа). Генератор гаммы детерминированный, поэтому на одинаковом ключе он выдаст один и тот же поток битов. Если зашифровать два разных текста одной гаммой, то при сложении двух шифртекстов гамма сократится (C1 XOR C2 = P1 XOR P2), и тексты можно будет вскрыть. Поэтому для обеспечения стойкости каждый раз берут новый сеансовый ключ или вектор инициализации.
5) Синхронизация отправителя и получателя. Так как шифрование идет побитово, при потере даже одного бита в канале связи вся остальная гамма сдвинется, и дальше пойдет мусор. Чтобы связь не рвалась, делают периодическую пересинхронизацию или используют самосинхронизирующиеся поточные шифры (где гамма зависит от нескольких последних принятых битов шифртекста).

4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового

режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

В качестве примера можно привести режим обратной связи (CFB) или режим счетчика (CTR).

Обоснование:

Если взять обычный режим простой замены (ECB) или сцепления блоков (CBC), то там блок открытого текста Pi подается прямо на вход функции блочного шифрования Ek: например, Ci = Ek(Pi). Чтобы получатель мог расшифровать блок Ci обратно, ему обязательно нужно применить обратную функцию Dk = Ek^(-1), то есть Pi = Ek^(-1)(Ci). А обратная функция существует только тогда, когда отображение Ek биективно (взаимно-однозначно). Если двум разным открытым блокам будет соответствовать один и тот же шифртекст, расшифровать его однозначно не получится.

В режиме обратной связи (CFB) блочный шифр используется по-другому - как генератор гаммы. На вход блочного шифра Ek подается не открытый текст, а содержимое верхнего регистра (на первом шаге это вектор инициализации IV, а на следующих шагах - предыдущий блок шифртекста C(i-1)). Полученный выход шифра складывается по модулю 2 (XOR) с открытым текстом:

Зашифрование: Ci = Pi XOR Ek(C(i-1)).

Когда получатель расшифровывает сообщение в режиме CFB, он вообще не использует обратную функцию расшифрования Dk. Вместо этого он берет тот же известный блок C(i-1) (или IV), прогоняет его через ту же самую прямую функцию зашифрования Ek, получает точно такую же гамму Ek(C(i-1)) и складывает ее по модулю 2 с принятым блоком шифртекста Ci:

Расшифрование: Ci XOR Ek(C(i-1)) = Pi XOR Ek(C(i-1)) XOR Ek(C(i-1)) = Pi.

Получается, что так как функция Ek и при шифровании, и при расшифровании запускается только в прямом направлении от одних и тех же данных, от нее требуется только детерминированность (выдавать одинаковый результат на одинаковом входе). Биективность

от Ek здесь не нужна, потому что однозначное восстановление текста Pi обеспечивается обратимостью самой операции XOR.

5. **Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.**

Решение:

В режиме гаммирования с обратной связью по шифртексту (CFB) открытый текст и шифртекст идут блоками по 8 бит (по 1 байту), а сам блочный шифр и его сдвиговый регистр обратной связи имеют размер 64 бита (вмещают 64 / 8 = 8 байтов).

На каждом шаге i очередной 8-битовый блок открытого текста Pi получается сложением по модулю 2 принятого байта шифртекста Ci с 8 битами гаммы Gi, которую выдал блочный шифр от текущего состояния 64-битового регистра: Pi = Ci XOR Gi. После этого регистр сдвигается на 8 бит, и байт Ci записывается в регистр.

Посмотрим, что произойдет при расшифровании, если в одном 8-битовом блоке шифртекста Ci исказился 1 бит:

1) На текущем шаге i в регистре еще лежат старые неискаженные данные, поэтому гамма Gi правильная. При сложении Pi = Ci XOR Gi ошибочный бит из Ci напрямую испортит ровно 1 бит в текущем 8-битовом блоке открытого текста Pi.
2) Дальше этот испорченный 8-битовый блок Ci попадает в 64-битовый сдвиговый регистр обратной связи. Так как длина регистра 64 бита, а сдвигается он по 8 бит за шаг, этот ошибочный байт будет идти по регистру еще 64 / 8 = 8 шагов, пока полностью из него не выйдет. На всех этих 8 шагах на вход блочного шифра подается искаженное значение регистра,

поэтому на выходе шифра гамма портится (из-за лавинного эффекта), и следующие 8 байтов открытого текста (от P(i+1) до P(i+8)) тоже расшифруются неправильно.

3) На 9-м шаге испорченный байт выталкивается из регистра, и дальше расшифрование снова идет нормально.

В итоге всего портится 1 + 8 = 9 подряд идущих 8-битовых блоков (9 байтов или 72 бита открытого текста).

Теперь переведем это в 64-битовые блоки открытого текста (в одном 64-битовом блоке ровно 8 байтов):

Если испорченный байт пришелся на самое начало 64-битового блока (1-й байт), то 9 ошибочных байтов займут все 8 байтов этого блока и 1-й байт следующего блока - то есть исказится 2 блока по 64 бита. Если же ошибка попала на k-й байт внутри первого 64-битового блока (со 2-го по 8-й байт), то в первом блоке испортится оставшиеся (9 - k) байт, а во втором блоке испортится первые k байт (что не превышает 8 байт, поэтому до третьего блока ошибка не дойдет). Значит, при любом положении ошибки цепочка из 9 байтов всегда задевает ровно два соседних 64-битовых блока.

Ответ: искажается 2 блока открытого текста по 64 бита.

6. **Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^(-9)? Обязательно должен быть приведен ход решения.**

Решение:

Хэш-функция выдает двоичные векторы длины n бит. Так как каждый бит может быть 0 или 1, всего существует M = 2^n разных значений хэш-функции.

В общей вероятностной модели коллизий (парадокс дней рождений) считается, что хэш-функция распределяет значения равномерно, а вероятность совпадения хэшей в выборке из k сообщений оценивается через количество пар k(k-1)/2, деленное на

число всех исходов 2^n. В нашем случае берется одна конкретная пара случайных сообщений (k = 2, число пар равно 1): первое сообщение дает какой-то хэш H(m1), а коллизия возникает, если хэш второго сообщения H(m2) совпадет с H(m1) (1 случай из 2^n возможных). Значит, вероятность коллизии для случайной пары сообщений равна:

P = 1 / 2^n.

По условию эта вероятность должна быть не больше 10^(-9):

1 / 2^n <= 10^(-9)

Перевернем дробь:

2^n >= 10^9

Прологарифмируем по основанию 2, чтобы найти n:

n >= log2(10^9)

n >= 9 * log2(10)

Посчитаем: log2(10) примерно равен 3,32.

Тогда n >= 9 * 3,32 = 29,88.

Так как длина вектора n должна быть целым числом, округляем в большую сторону и получаем n = 30 (при n = 29 будет 2^29 = 536 870 912 < 10^9, а при n = 30 будет 2^30 = 1 073 741 824 > 10^9).

Ответ: при n >= 30 (минимальное n = 30).